

Object-Oriented Drawing Editor

Project Report

Author: Ali Ayhan Gunder - 2021400219

Course: Human Computer Interaction CMPE 496

Date: June 30, 2025

Contents

1	Executive Summary	2
2	Project Overview	2
2.1	Purpose	2
2.2	Key Features	2
3	Technical Architecture	2
3.1	Class Hierarchy	2
3.2	Object-Oriented Principles	3
4	Technical Implementation	3
4.1	Graphics Rendering	3
4.2	Event Handling	3
4.3	Mathematical Algorithms	3
5	User Interface Design	3
6	Performance and Quality	4
6.1	Strengths	4
6.2	Educational Value	4
7	Future Enhancements	4
8	Conclusion	4

1. Executive Summary

This project implements a comprehensive drawing application in Java using the Swing GUI framework. The application demonstrates fundamental object-oriented programming principles including inheritance, polymorphism, encapsulation, and abstraction through an interactive graphics editor that allows users to draw, select, move, and erase geometric shapes.

2. Project Overview

2.1. Purpose

The Object-Oriented Drawing Editor serves as both a functional graphics application and an educational demonstration of OOP principles, providing tools to create circles, squares, and lines while showcasing abstract classes, inheritance hierarchies, and polymorphic behavior.

2.2. Key Features

- Interactive shape creation (Circle, Square, Line)
- Dynamic color selection through dropdown menu
- Shape selection and movement via drag-and-drop
- Shape deletion with erase tool
- Visual feedback for selected shapes
- Clean, intuitive user interface

3. Technical Architecture

3.1. Class Hierarchy

The application follows a well-structured inheritance hierarchy:

- **Abstract Shape Class:** Foundation for all drawable objects with common properties and abstract methods
- **Circle:** Implements circular geometry with radius-based containment testing
- **Square:** Implements rectangular geometry with bounding box collision detection
- **Line:** Implements linear geometry with perpendicular distance-based selection
- **DrawingPanel:** Custom JPanel handling mouse events and shape rendering
- **hw1:** Main application class managing UI components and user interactions

3.2. Object-Oriented Principles

- **Encapsulation:** Private variables with controlled getter/setter methods
- **Inheritance:** Clear parent-child relationships with proper `super()` calls
- **Polymorphism:** Runtime method resolution and uniform shape treatment
- **Abstraction:** Abstract methods enforcing consistent behavior across subclasses

4. Technical Implementation

4.1. Graphics Rendering

The application utilizes Java's Graphics2D API with anti-aliasing, custom styling, and visual feedback for selected shapes.

4.2. Event Handling

Sophisticated mouse event processing includes context-sensitive behavior for shape creation, selection, and movement with real-time delta calculations.

4.3. Mathematical Algorithms

Circle Containment Detection:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \leq radius \quad (1)$$

Line Proximity Detection:

$$distance = \frac{|Ax + By + C|}{\sqrt{A^2 + B^2}} \quad (2)$$

5. User Interface Design

The interface uses BorderLayout for main organization and FlowLayout for toolbar components, providing:

- Visual tool selection with button highlighting
- Contextual instruction updates
- Intuitive drag-and-drop manipulation
- Color-coded selection indicators
- Consistent interaction patterns

6. Performance and Quality

6.1. Strengths

- Efficient shape collection management using ArrayList
- Optimized hit detection with reverse iteration
- Smooth graphics rendering with anti-aliasing
- Minimal object creation during runtime
- Consistent naming conventions and documentation

6.2. Educational Value

The project demonstrates key computer science concepts:

- Design patterns (Template Method, Observer, Strategy)
- Data structures (dynamic arrays, hash maps)
- Algorithms (geometric calculations, collision detection)
- Event-driven programming paradigms

7. Future Enhancements

- Additional shape types (triangles, polygons, curves)
- Save/load functionality for persistent storage
- Undo/redo operations with command pattern
- Keyboard shortcuts and accessibility features
- Zoom and pan capabilities
- Shape grouping and layering

8. Conclusion

The Object-Oriented Drawing Editor successfully demonstrates OOP principles while providing a functional graphics application. The clean architecture, comprehensive feature set, and extensible design make it an excellent example of how theoretical concepts translate into practical software solutions.

The project showcases professional software development practices including proper code organization, documentation, and user interface design. It effectively balances educational objectives with practical functionality, creating a robust foundation for understanding object-oriented design principles.